MARS-OPS

Autonomous Mars Colony Management & Emergency Simulator

Discussion Guide

Full walkthrough of the code and the UML diagrams
Expected discussion questions with model answers
Known weaknesses and how to talk about them honestly

Prog II - Lab 5 | Java project, discussion preparation

Contents

1 The 60-second pitch and how the whole project fits together
2 Code tour: every package and class, and why it exists
3 The four OOP pillars - with real examples from the code
4 UML relationships: association, aggregation, composition, inheritance
5 Missions in depth: submission, the lifecycle and failure paths
6 Resources in depth: totals, allocation policy, all-or-nothing
7 The Mars environment
8 Emergencies in depth (oxygen leak, fire, rover breakdown, medical)
9 Simulation and time
10 The five UML diagrams: how to explain each one
11 The five demonstration scenarios and a live-demo script
12 Testing
13 Honest limitations and what you would improve
14 Expected questions and model answers (about 50)
15 Cheat sheet: names, numbers and rules to memorise

How to use this guide. Sections 1-13 teach you the project so you can explain it in your own words. Section 14 is practice for the questions you are likely to get. Section 15 is a one-page summary for the last look before you walk in. Green boxes are things worth *saying out loud*; orange boxes are traps and weak spots - know them before the examiner finds them.

Important about this guide's accuracy. Everything here was written from the code as it is now (final code in `src/`) and from the diagrams in `mars colony (4).drawio`. Class and method names are copied exactly, including spelling quirks such as `EnviromentStates` and `COMMUNICATION_DISTRIBUTION`. The GUI is explained only briefly, as requested.

1. The 60-second pitch and the big picture

> **Say this first.** "MARS-OPS simulates a Mars colony. The **Colony** object coordinates everything: it holds the people, robots, facilities, missions and emergencies. The interesting part is that the colony never checks what *type* an object is. It asks the object what it can do: `agent.canPerform(mission)`, `agent.canHandleEmergency(type)`, `environment.getMissionRestrictionReason(...)`. Each mission and emergency subclass defines its own rules and owns its own state machine. Time advances through `Simulation.advanceTime(colony, n)`. We show five scenarios: a successful exploration, a rejected mission, a resource shortage, an oxygen-leak emergency and a dust storm."

1.1 What the system does (the six jobs from the specification)

Job in the spec	Where it lives in the code
Manage personnel and robotic units	`Personnel` (+ `Scientist`, `Engineer`, `MedicalOfficer`), `RoboticUnit` (+ `ExplorationRover`, `RepairRobot`, `Drone`); added with `Colony.addPersonnel` / `addRobot`
Manage habitats/facilities, track resources	`Facilities` (+ `Habitat`, `Laboratory`, `StorageFacility`, `PowerStation`, `CommunicationStation`); `Colony.getColonyResourceTotal()`
Assign missions, simulate time	`Colony.submitMission`, `startActiveMission`; `Simulation.advanceTime`
Decide whether entities can do missions	`Agent.canPerform`, `Mission.hasValidTeamComposition`, `MarsEnvironment.getMissionRestrictionReason`
Handle failures and emergencies, update colony state	`Colony.handleEmergency`, the `Emergency` hierarchy, `Mission.failMission`
Produce a status report	The dashboard in the admin GUI (colony-wide totals, active mission, time) and the printed before/after snapshots in `Main` (see limitations)

1.2 Architecture at a glance

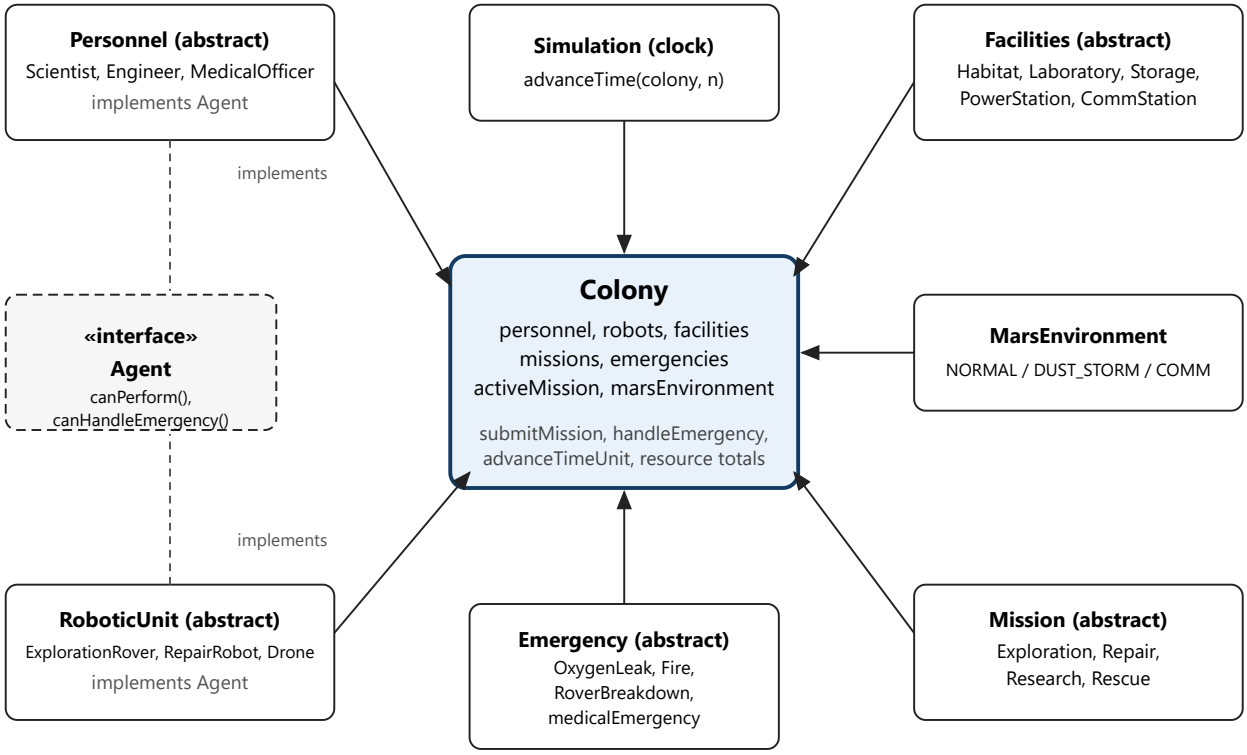


Figure 1 - the colony in the middle; every other box is something the colony owns or asks questions of.

1.3 One idea that explains most of the design

Look at `Colony.submitMission(...)`. It never contains `if (agent instanceof Engineer)`. It asks four different objects four different questions and each answers for itself:

```
agent.isAvailable()  /  agent.canPerform(mission)        ← the AGENT knows its own skills and
capabilities
mission.hasValidTeamComposition(agents)                   ← the MISSION knows what team it needs
marsEnvironment.getMissionRestrictionReason(mission, agents) ← the ENVIRONMENT knows what it
forbids
mission.checkresources(getColonyResourceTotal())          ← the MISSION knows what it costs
```

> **Why this matters.** Adding a new mission type (say `Mining`) means writing one new subclass and overriding `hasValidTeamComposition`. The colony code does not change. That is the practical payoff of abstraction and polymorphism, and it is exactly what section 6.1 of the specification asks for.

2. Code tour - package by package

Source folder: `src/`. The model packages are listed first; the GUI packages are last and only summarised.

2.1 `enums` - the vocabulary of the system

Enum	Values	Used for
`Skills`	MECHANICAL, SCIENTIFIC, MEDICAL, COMMUNICATION, FIRE_RESPONSE	What a **person** can do
`Capability`	SURFACE_TRAVEL, EQUIPMENT_CARRYING, EXPLORATION, EQUIPMENT_REPAIR, FACILITY_OPERATIONS, AERIAL_RECONNAISSANCE, ACCESS_INACCESSIBLE_AREAS	What a **robot** can do
`Resources`	OXYGEN, WATER, FOOD, FUEL, BATTERY_POWER, MEDICAL_SUPPLIES, SPARE_PARTS	The seven resources of section 5 of the spec
`OperationalStatus`	OPERATIONAL, DAMAGED, UNAVAILABLE, DESTROYED	Facility and equipment condition (`isOperational()` helper)
`Mission_Emergency_States`	ASSIGNED, INPROGRESS, COMPLETED, FAILED	Mission state. (Name says "Emergency" too, but emergencies use `EmergencyStatus`.)
`EmergencyStatus`	PENDING, RESOLVED, UNRESOLVED	Emergency state
`EmergencyType`	OXYGEN_LEAK, FIRE, ROVER_BREAKDOWN, MEDICAL	Lets an agent answer `canHandleEmergency(type)`
`EnviromentStates`	DUST_STORM, COMMUNICATION_DISTRIBUTION, NORMAL_CONDITION	Environment (spelling is as in the code)
`FacilityType`	HABITAT, LABORATORY, STORAGE, POWER_STATION, COMMUNICATION_STATION	Set once by each facility subclass in its constructor

> **Quirk to know.** A mission that has been created but not yet accepted has `state == null`. There is no "NEW" value in the enum. The state machine diagram draws it as a **NEW** state; in code it is `null`.

2.2 `agents` - the capability contract

`Agent` is an interface implemented by both `Personnel` and `RoboticUnit`, which share no parent class.

Method	Meaning
`canPerform(Mission)`	Can this agent take part in this mission? (abstract)
`canHandleEmergency(EmergencyType)`	Can this agent respond to this emergency type? (abstract)
`isAvailable()`, `makeAvailable()`, `setAvailability(boolean)`	Availability; `makeAvailable()` also clears the agent's mission

`assignMission(Mission)`	Called by `Mission.assign` to record the mission on the agent
`getCurrentLocation()`, `setCurrentLocation(String)`	Every agent has a location (spec rule 9.1.1)
`hasSkill`, `hasCapability` (default `false`)	Used by missions and the environment to inspect a team without knowing concrete types
`hasEnoughBatteryFor`, `consumeBatteryForTimeUnit` (defaults `true`)	People ignore battery; robots override these

> **Why default methods?** People have no battery, so a default that returns `true` lets the mission code call `consumeBatteryForTimeUnit()` on *every* agent without a type check. Robots override it.

2.3 `personnel`

`Personnel` (abstract, implements `Agent`) holds: `name, id, healthStatus, currentLocation, skills (Set<Skills>), fatigueLevel (0-100), availability, mission`. The constructor validates everything (non-blank name, id > 0, fatigue between 0 and 100, non-empty skills, no null skills) and refuses a person that is *available while assigned to a mission*. The base `canPerform` only checks "mission not null and available"; subclasses add the skill rules.

Subclass	`canPerform(mission)` is true when ...
`Scientist`	available and (`Research` or `Exploration` with skill SCIENTIFIC, or `Rescue` with skill COMMUNICATION)
`Engineer`	available and (`Repair` with skill MECHANICAL, or `Rescue` with skill COMMUNICATION)
`MedicalOfficer`	available and `Rescue` with skill COMMUNICATION. Also has an overload `canPerform(medicalEmergency)`.

`Personnel.canHandleEmergency(type)`: must be available, then OXYGEN_LEAK needs MECHANICAL, FIRE needs FIRE_RESPONSE, ROVER_BREAKDOWN needs MECHANICAL, MEDICAL needs MEDICAL. Notice it checks **skills**, not the subclass - so an Engineer who also has the MEDICAL skill could answer a medical emergency.

2.4 `roboticunit`

`RoboticUnit` (abstract, implements `Agent`): `batteryLevel (0-100), healthStatus, currentLocation, availability, capabilities, mission, equipment`. Battery cost is a constant `BATTERY_COST_PER_TIME_UNIT = 1.0`.

- `canPerform(mission)`: available **and** battery ≥ `mission.neededTimeUnits × 1.0` **and** the robot's capabilities contain **all** of the mission's required capabilities.
- `consumeBatteryForTimeUnit()`: returns `false` if battery < 1, else subtracts 1.
- `canHandleEmergency(type)`: only OXYGEN_LEAK and ROVER_BREAKDOWN, and only if available, battery > 0 and it has EQUIPMENT_REPAIR. Fire and medical are always `false` for robots.
- `hasOperationalEquipment()`: used by the Exploration team rule.

Subclass	Capabilities
`ExplorationRover`	Passed in by the creator (the demo uses SURFACE_TRAVEL, EQUIPMENT_CARRYING, EXPLORATION)

`RepairRobot`	Fixed: EQUIPMENT_REPAIR, FACILITY_OPERATIONS
`Drone`	Fixed: AERIAL_RECONNAISSANCE, ACCESS_INACCESSIBLE_AREAS

2.5 `facilities`

`Facilities` (abstract): `type (final), location, capacity, operationalStatus, resources (Map<Resources,Double>), equipment (List<Equipment>), occupants (List<Agent>)`. Key methods: `getResourceAmount(r)`, `deductResource(r, amount)` (removes *at most what exists* and returns the amount actually removed, so a resource can never go negative), `addResource`.

Subclass	What it adds
`Habitat`	**Composition:** `private final LifeSupportSystem lifeSupport` created in its own constructor. `applyOxygenLoss(amount)` deducts oxygen from this habitat only.
`Laboratory`	`onFire`; `catchFire()` → UNAVAILABLE, `extinguishFire()` → DAMAGED, `repair()` → OPERATIONAL
`StorageFacility`	`getTotalStored()`, `store(r, amount)` (refuses if not operational or over capacity)
`PowerStation`	`outputPerTimeUnit`, `generatePower()` (exists but the simulation does not call it)
`CommunicationStation`	`isCommunicationAvailable()` (exists but the environment rules do not use it)

`Equipment`: name, `OperationalStatus`, and a `Category` (GENERAL or SCIENTIFIC). The category is how a `Research` mission finds "scientific equipment". `LifeSupportSystem`: a single `functional` flag.

2.6 `missions`

`Mission` (abstract) is the heart of the lifecycle. Fields: `agents, neededtimesunits, elapsedTimeUnits, facility, state, requiredResources, requiredCapabilities, failureReason, colony, assigningAgent`.

Method	Role
`assign(agent)`	Validates (colony link, state, duplicates, availability, capability), calls `agent.assignMission(this)`, adds the agent, sets state ASSIGNED
`unassign(agent)`	Only while ASSIGNED. If no agents are left the state goes back to `null`; if the team becomes invalid the mission fails
`rejectMission(reason)`	For missions with `state == null`: sets FAILED + reason. This is the "rejected, never active" path
`startMission()`	ASSIGNED → INPROGRESS; checks agent battery, asks the colony to reserve resources (`prepareMissionStart`); fails the mission if either check fails
`advanceTimeUnit()`	Only when INPROGRESS: each agent consumes battery (if one cannot, mission fails), `elapsed++`, completes when `elapsed ≥ needed`
`finishMission(boolean)`	INPROGRESS → COMPLETED (runs hook `onSuccessfulCompletion()`) or FAILED; releases agents
`failMission(reason)`	From ASSIGNED or INPROGRESS → FAILED; releases agents; tells the colony so it clears the active mission
`checkresources(map)`	True if every required resource is covered by the given totals
`hasValidTeamComposition(agents)`	Base: non-empty. **Overridden** by every subclass

Subclass	Extra data	Team rule (`hasValidTeamComposition`)	On success
`Exploration`	`region`	a SCIENTIFIC agent **and** a robot with SURFACE_TRAVEL + EQUIPMENT_CARRYING + operational equipment	-
`Repair`	`repairObject` (Equipment or Facilities, must be DAMAGED)	a MECHANICAL agent **and** an agent with EQUIPMENT_REPAIR	`performRepair()` sets the object OPERATIONAL
`Research`	`topic`, `experimentCompleted`	a SCIENTIFIC agent **and** an OPERATIONAL `Laboratory` with operational SCIENTIFIC equipment	`experimentCompleted = true`
`Rescue`	-	a COMMUNICATION-skilled person **and** a vehicle (SURFACE_TRAVEL or ACCESS_INACCESSIBLE_AREAS)	-

2.7 `emergencies`

`Emergency` (abstract): `type, status (starts PENDING), reason, failureReason, location (Facilities or String), agents, requiredResources, occurrenceRegistered`.

- **Template method:** `registerOccurrence()` is `final`; it runs once and calls the overridable hook `onOccurrence()`. `markUnresolved()` calls the hook `onUnresolved()` the first time only.
- `hasRequiredResponders(responders)`: base rule is "at least one responder says it can handle this type"; `OxygenLeak` overrides it.
- `requiresOperationalFacility()`: base `false`; `medicalEmergency` returns `true`.
- `applyResponse()`: abstract - what the response actually does.

Subclass	onOccurrence	onUnresolved	applyResponse	Resources
`OxygenLeak` (habitat)	habitat loses `oxygenLossAmount`	habitat loses it **again**	`repaired = true`	1.0 SPARE_PARTS
`Fire` (laboratory)	`catchFire()` (lab UNAVAILABLE)	-	`extinguishFire()` (lab DAMAGED)	none
`RoverBreakdown` (surface)	active mission of that rover fails ("Rover breakdown"); rover DAMAGED and unavailable	-	returns `true`	none
`medicalEmergency` (patient)	-	-	patient's health becomes "Stable"	1.0 MEDICAL_SUPPLIES

2.8 `colony`, `marsenvironment`, `simulation`

`Colony` is covered step by step in sections 5, 6 and 8. **`MarsEnvironment`** in section 7 and **`Simulation`** in section 9.

2.9 The GUI packages (brief)

- `admingui` (newest): `AdminGuiMain` opens `AdminConsole` - a dashboard, tabs for personnel, facilities, robots, missions and emergencies, an animated colony view, a live activity feed, and sound

(`SoundManager`). `SimulationContext` holds the live colony and broadcasts change events to the panels. `SampleColonyFactory` builds a ready-made colony. No simulation rules live here - the panels call the same public methods the console demo calls.

- `gui` (older): `AdminDashboard`, `ScenarioLabPanel`, `AppSession`, `Main` - an earlier version of the same idea.
- `Main.java` (default package): the console demo of the five required scenarios - this is the "testing main class" required by the deliverables.
- `tests/MissionAssignmentTest.java`: four hand-written checks (section 12).

3. The four OOP pillars - with real examples

3.1 Encapsulation

Fields are `private`; the only way to change state is through methods that validate it.

- `Personnel` constructor rejects blank names, ids ≤ 0, fatigue outside 0-100, empty or null skills. `setAvailability(true)` throws if the person is still assigned to a mission.
- Getters return **copies**: `Colony.getMissions()` returns `new ArrayList<>(this.missions)`, `Facilities.getResources()` returns a new `HashMap`. Outside code cannot edit the colony's internal lists.
- State transitions are guarded inside the owner: `Mission.failMission` throws `IllegalStateException` unless the mission is ASSIGNED or INPROGRESS; `Mission.setRequiredResources` refuses after the mission has been assigned.
- `Facilities.deductResource` can never push a quantity below zero because it removes `Math.min(available, requested)`.

> **Say:** "Encapsulation is not only private fields. The important part is that each object protects its own invariants: a resource cannot go negative, a person cannot be available and on a mission at the same time, a failed mission cannot be restarted."

3.2 Abstraction

- `Agent` hides *whether* something is a human or a robot behind `canPerform` / `canHandleEmergency`.
- `Mission`, `Emergency`, `Facilities`, `Personnel`, `RoboticUnit` are abstract: they define what is common and force subclasses to supply the specifics (`Emergency.applyResponse()` is abstract).
- The colony deals with the abstract type: `List<Agent>`, `Mission`, `Emergency`, `Facilities`.

3.3 Inheritance

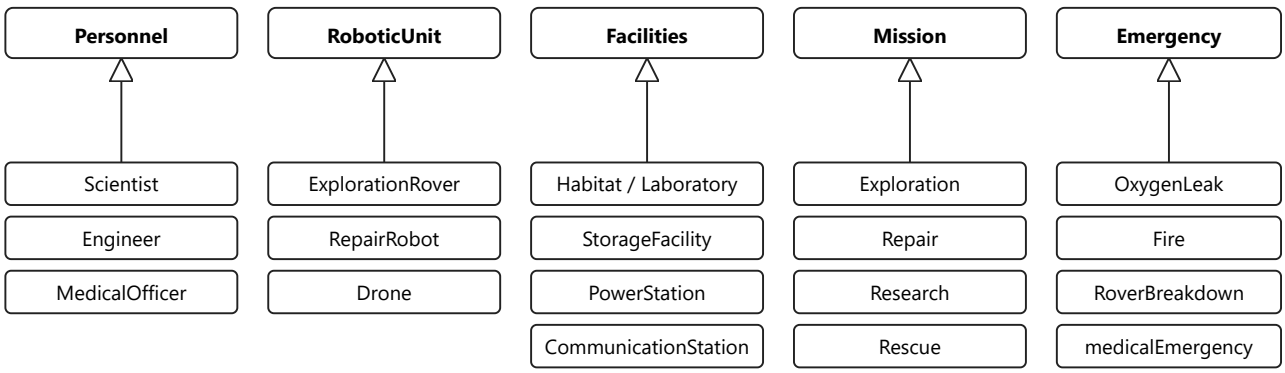


Figure 2 - the five inheritance trees. (Habitat and Laboratory share the same box here to save space; they are two separate subclasses.)

Why inheritance is right here: a `Habitat` *is a* `Facilities`; an `Engineer` *is a* `Personnel`. The subtypes reuse the parent's fields and validation, and override only what differs.

3.4 Polymorphism

Call site (parent type)	What actually runs

`agent.canPerform(mission)` in `Colony.submitMission`	`Scientist`, `Engineer`, `MedicalOfficer` or `RoboticUnit` version
`mission.hasValidTeamComposition(agents)`	The `Exploration`, `Repair`, `Research` or `Rescue` version
`emergency.hasRequiredResponders(responders)`	Base version, or `OxygenLeak`'s "human + robot" version
`emergency.applyResponse()`	Repair the leak / extinguish the fire / treat the patient / nothing extra for a rover
`emergency.registerOccurrence()` → `onOccurrence()`	Oxygen deduction / `catchFire` / mission failure
`mission.finishMission(true)` → `onSuccessfulCompletion()`	Perform the repair / mark the experiment complete

> **Best single example to quote:** "`Colony.handleEmergency` works for all four emergency types with the *same code*. It calls `registerOccurrence()`, `hasRequiredResponders()` and `applyResponse()` on an `Emergency` reference; the object decides what those mean."

4. UML relationships: what was chosen and why

The specification asks you to explain **why** each relationship was chosen and how it appears in the UML and in Java.

Relationship	Where	Why this and not another	In Java	In UML (class diagram)
Inheritance	Subtypes of `Personnel`, `RoboticUnit`, `Facilities`, `Mission`, `Emergency`	True "is-a"; subtypes override behaviour	`extends`	Solid line, hollow triangle at the parent
Realization	`Personnel` and `RoboticUnit` → `Agent`	Two unrelated hierarchies must be treated alike	`implements`	Dashed line, hollow triangle
Composition	`Habitat` ◆ `LifeSupportSystem`	The life-support system exists only as part of its habitat; created and owned by it	`private final LifeSupportSystem lifeSupport = new ...` in the constructor	Filled diamond at `Habitat`
Aggregation	`Colony` ◇ personnel, robots, facilities, missions, emergencies; `Facilities` / `RoboticUnit` ◇ `Equipment`	A whole made of parts that exist independently; a person or an item of equipment can exist without this colony	List fields filled by `addPersonnel`, `addRobot`, `addFacility`, `setEquipment`	Hollow diamond at the whole
Association	`Mission` - `Agent`; `Mission` - `Facilities`; `Emergency` - `Facilities` / `Agent`; `Colony` - `MarsEnvironment`	Each side can exist without the other; they only "work with" each other	Reference fields (`Mission.agents`, `Mission.facility`, `Colony.marsEnvironment`)	Plain line / open arrow

Likely follow-up: "Why is Colony-to-Mission aggregation and not composition?" Because missions are created outside the colony and handed to `submitMission`, and a rejected mission still exists as an object afterwards. Composition would require the colony to create and own them. Same reasoning for personnel: a `Scientist` is built first, then added with `addPersonnel`.

Likely follow-up: "Is LifeSupportSystem really used?" Honest answer: the object is created and owned (composition is real), but its `functional` flag is not yet used by the oxygen-leak logic. It is there to show the relationship; wiring it into emergencies would be a natural extension.

4.1 Where responsibilities sit (Single Responsibility)

Class	Is responsible for	Is **not** responsible for
`Colony`	Coordination: one active mission, availability check, resource allocation across facilities, dispatching responders	Knowing what a mission needs, or what an emergency does

`Mission`	Its own state machine, team rule, duration, release of agents	Choosing which facility supplies resources
`Emergency`	Its own status, responder rule, effect on the world	Finding responders or consuming resources
`Agent` implementors	Answering "can I?" about missions and emergencies	Knowing about other agents
`MarsEnvironment`	Saying what the current condition forbids or breaks	Failing missions itself - it only returns a reason
`Facilities`	Holding and deducting its own resources safely	Scheduling missions (the spec says facilities do not own missions)

5. Missions in depth

5.1 Step-by-step: `Colony.submitMission(mission, agents)`

Returns `true` only if the mission was accepted. Checks run in this exact order and stop at the first failure.

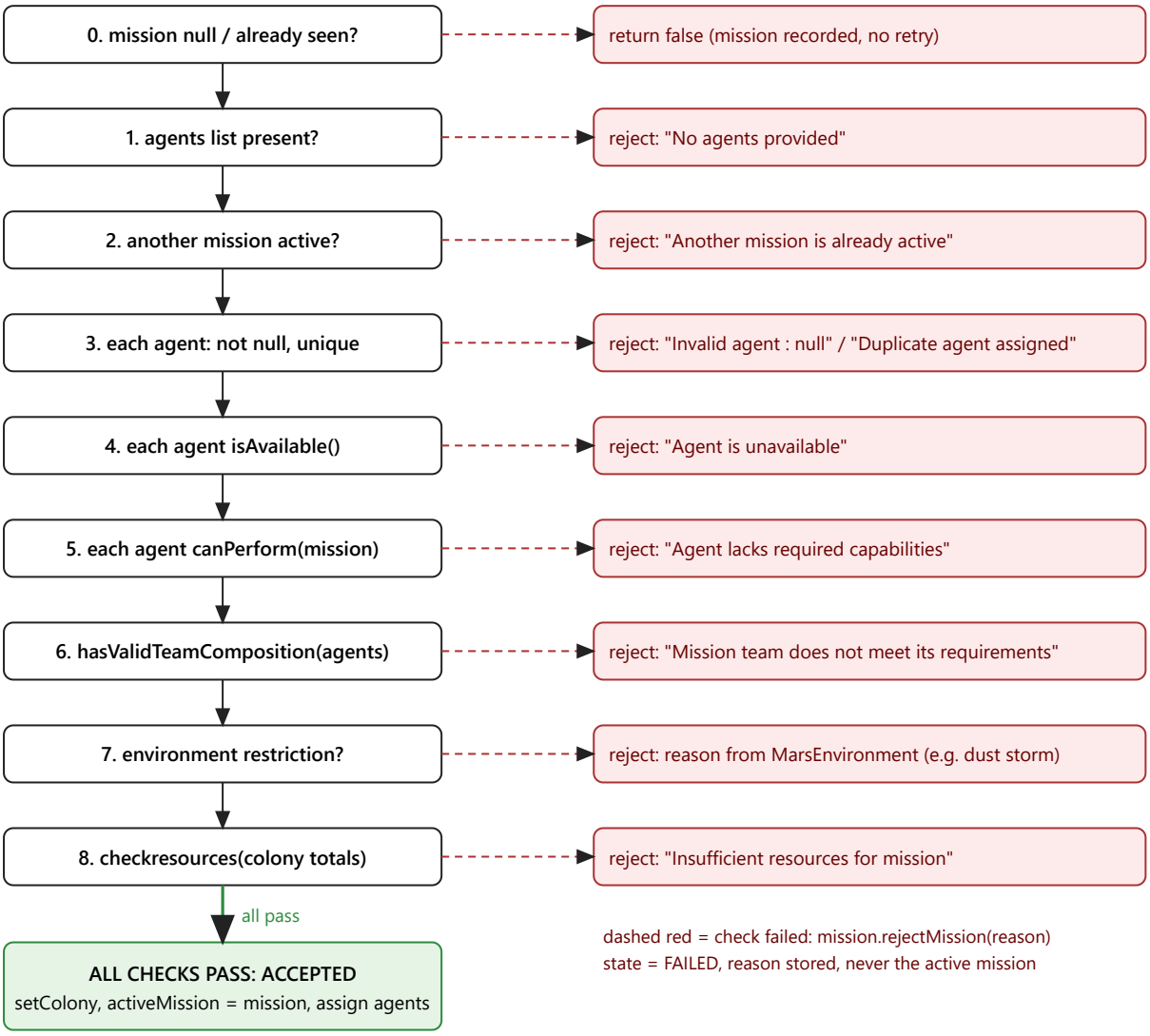


Figure 3 - the submission checks in code order. Every red branch calls `mission.rejectMission(reason)` : state = FAILED, reason stored, never active.

Details worth knowing:

- **No retry.** At the top of `submitMission` : if `mission.getState() != null` the mission is simply recorded and `false` is returned. A mission that has already been processed (failed, completed) can never be submitted again - exactly spec rule 6.2.6.
- **Rejected missions are still recorded** in `colony.missions` so the operator can inspect the reason later (use case "Inspect mission outcomes"). They are never `activeMission` .
- **Accepted path:** `mission.setColony(this)` , `activeMission = mission` , then `mission.assign(agent)` for each agent. If an assignment throws, the colony **rolls back**: unassigns the ones already assigned, clears `activeMission` , and rejects the mission with "Mission assignment failed: ...". The test `testRollback` checks this.

- **Mutual call guard.** `Mission.assign` calls `agent.assignMission(this)`, and a person's `assignMission` calls `mission.assign(this)` if it was called from outside. The field `Mission.assigningAgent` breaks that loop: `agent.assignMission` only proceeds when `mission.isAssigningAgent(this)` is true.

5.2 The mission lifecycle

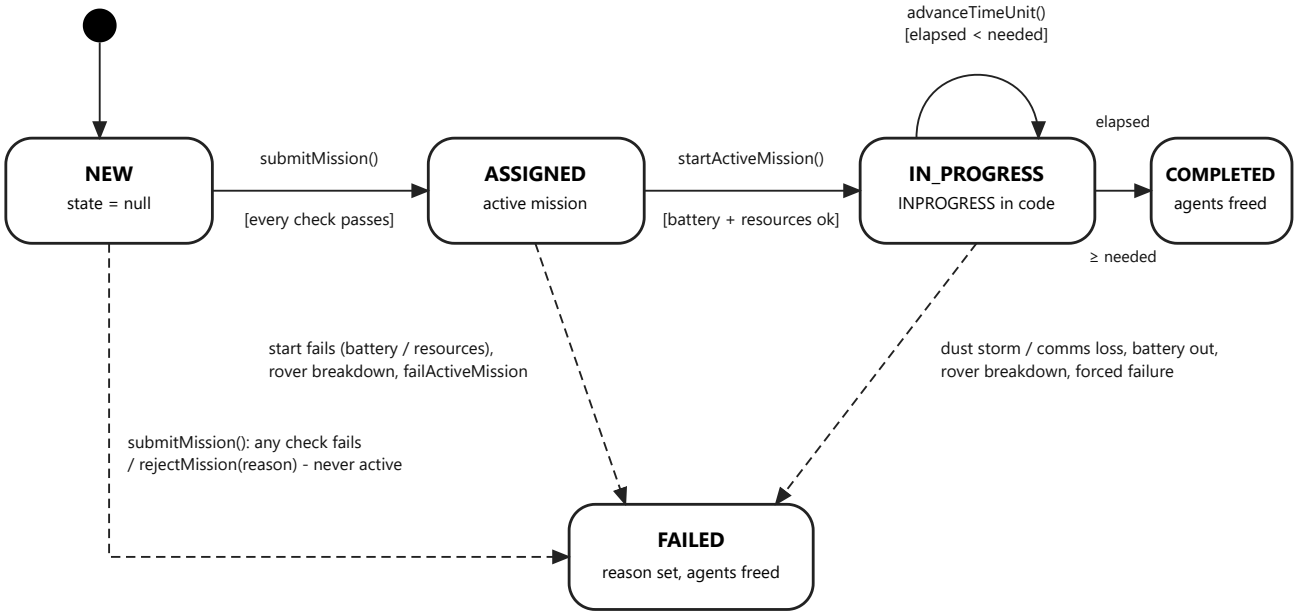


Figure 4 - mission states (solid = normal progress, dashed = failure transitions). COMPLETED and FAILED are final; the diagram page "State Machine Diagram" in the .drawio file has every guard and action.

Transition	Triggered by	Guard	Action
NEW → ASSIGNED	`Colony.submitMission`	all eight checks pass	`assign` each agent, `activeMission = mission`
NEW → FAILED	`Colony.submitMission`	any check fails	`rejectMission(reason)`
ASSIGNED → NEW	`Mission.unassign`	the last agent is removed	state = null, colony active mission cleared
ASSIGNED → INPROGRESS	`Colony.startActiveMission()`	every agent has enough battery; resources reserved	deduct resources, `elapsed = 0`
ASSIGNED → FAILED	`startMission` or `failActiveMission` or a rover breakdown	battery too low; resources gone; forced	`failMission(reason)`
INPROGRESS → INPROGRESS	`Mission.advanceTimeUnit`	`elapsed < needed`	`elapsed++`, robots lose 1 battery
INPROGRESS → COMPLETED	`Mission.advanceTimeUnit` (or `finishActiveMission(true)`)	`elapsed ≥ needed`	`onSuccessfulCompletion()`, release agents
INPROGRESS → FAILED	environment check, battery out, breakdown, `finishActiveMission(false)`, `failActiveMission`	the stated condition	`failMission(reason)`, release agents

> **Points the examiner may probe.** (1) **ASSIGNED does not become INPROGRESS by itself.** Starting is an explicit call, `Colony.startActiveMission()`; the clock does not start missions. If time advances while a mission is only ASSIGNED, it waits (`Mission.advanceTimeUnit` returns immediately unless INPROGRESS) - although an environment failure can still fail it. (2) The spec says "a mission starts counting when it becomes IN_PROGRESS" - that is why `elapsedTimeUnits` is reset to 0 in `startMission`. (3) `Colony.finishActiveMission(true/false)` exists as a manual override (the GUI's "Force Finish" buttons); in normal runs completion comes from time advancing.

5.3 How "all assigned agents become available again" is guaranteed

Every terminal path goes through `failMission`, `rejectMission` or `finishMission`, and each one loops over `agents` calling `agent.makeAvailable()` and then clears the list. Each also calls `notifyColonyIfTerminal()`, which makes the colony clear `activeMission`, so a new mission can be submitted straight away.

6. Resources in depth

6.1 Rules (section 5.1 of the specification) and where each is implemented

Spec rule	Implementation
1. Colony-wide total = sum over **operational** facilities	`Colony.getColonyResourceTotal()` skips every facility whose status is not OPERATIONAL
2. May draw from more than one facility	`prepareMissionStart` and `consumeEmergencyResources` build a *deduction plan* spanning facilities
3. All-or-nothing	The plan is built first without touching anything; if `remaining` still has a positive amount the method returns `false` and nothing is deducted. During the actual deduction, any mismatch or exception calls `restoreResources(originalResources)`
4. Oxygen leak is local	`Habitat.applyOxygenLoss` deducts from that habitat only
5. Report shows colony-wide totals	Dashboard resource bars use `getColonyResourceTotal()`
6. Allocation policy documented	README section 5 (and below)

6.2 The allocation policy you chose

> **Say:** "First-fit in the order the facilities were added to the colony. For each resource we take as much as we can from the first operational facility, then move to the next, until the request is covered. If the colony as a whole cannot cover it, we take nothing."

Worked example (the one in the spec): Habitat A has 300 oxygen, Storage has 200. A request for 450: plan = 300 from A, 150 from Storage → valid. A request for 600: after both, 100 remains → nothing is deducted, the operation fails.

Why first-fit? It is deterministic, simple to explain, easy to test, and the spec says the policy is a design decision to be documented. Alternatives (take from the fullest facility, from the nearest one, proportionally) would work too; they only change the loop that builds the plan.

6.3 When are mission resources consumed?

1. **At submission** - only *checked*: `mission.checkresources(colony totals)`. A mission that cannot be afforded is rejected immediately (spec 6.2 rule 3).
2. **At start** (`startMission` → `colony.prepareMissionStart`) - actually *deducted*, once. The set `resourcesPreparedMissions` (an identity set) makes sure the same mission is never charged twice.
3. If stock vanished between submission and start (e.g. a facility was damaged), the mission fails with "Insufficient resources to start mission" and nothing is consumed.

> **Be ready to defend:** "Why not deduct at submission?" Answer: the spec says a mission is ASSIGNED and "remains active until it completes or fails"; an assigned mission that never starts should not have burnt resources. Consuming when execution begins also matches "a repair mission might consume 2 spare parts" - consumption belongs to doing the work.

6.4 Scenario 3 in numbers

Habitat-C has 1.0 oxygen, the mission needs 5.0. Step 8 of the submission checks fails. Result: `FAILED`, reason "Insufficient resources for mission", mission not active, oxygen still 1.0 before and after - never negative, nothing consumed.

7. The Mars environment

`MarsEnvironment` holds one `EnviromentStates` value. It never touches missions; it only **answers questions** with a reason string or `null`.

Method	Asked by	When
`getMissionRestrictionReason(mission, agents)`	`Colony.submitMission` (check 7)	Before a mission is accepted
`getActiveMissionFailureReason(mission)`	`Colony.advanceTimeUnit`	Every time unit, **before** the mission is progressed

Condition	At submission	For an active mission (next time unit)
NORMAL_CONDITION	no restriction	no effect
DUST_STORM	rejects an `Exploration` mission that uses an agent with SURFACE_TRAVEL: "Dust storm prevents exploration missions from starting"	fails any active mission with a SURFACE_TRAVEL agent: "Dust storm caused the active rover mission to fail"
COMMUNICATION_DISTRIBUTION	rejects a mission using a drone (AERIAL_RECONNAISSANCE): "Communication disruption prevents remote drone control"; rejects a `Rescue` mission with a COMMUNICATION agent	fails an active mission with a drone, or a `Rescue` with a communication agent

The README question: "What happens to missions already in progress when conditions change?" Nothing happens at the instant the condition is changed. At the **next** time unit the colony asks the environment about the active mission; if it returns a reason, the colony fails the mission immediately (`failActiveMission(reason)`), the agents are released and the reason recorded. A mission the condition does not affect (e.g. a research mission during a dust storm) simply continues.

Subtle points. (1) Because a `Rescue` always needs a COMMUNICATION-skilled person, every Rescue is effectively blocked during a communication disruption. (2) A dust storm rejects at *submission* only `Exploration` missions, but it fails *active* missions of any type that include a rover. So a Rescue with a rover can be accepted during a storm but would fail on the next time unit. (3) `CommunicationStation.isCommunicationAvailable()` exists but is not consulted by the environment rules. Emergencies are not affected by the environment.

8. Emergencies in depth

8.1 The common procedure: `Colony.handleEmergency(emergency)`

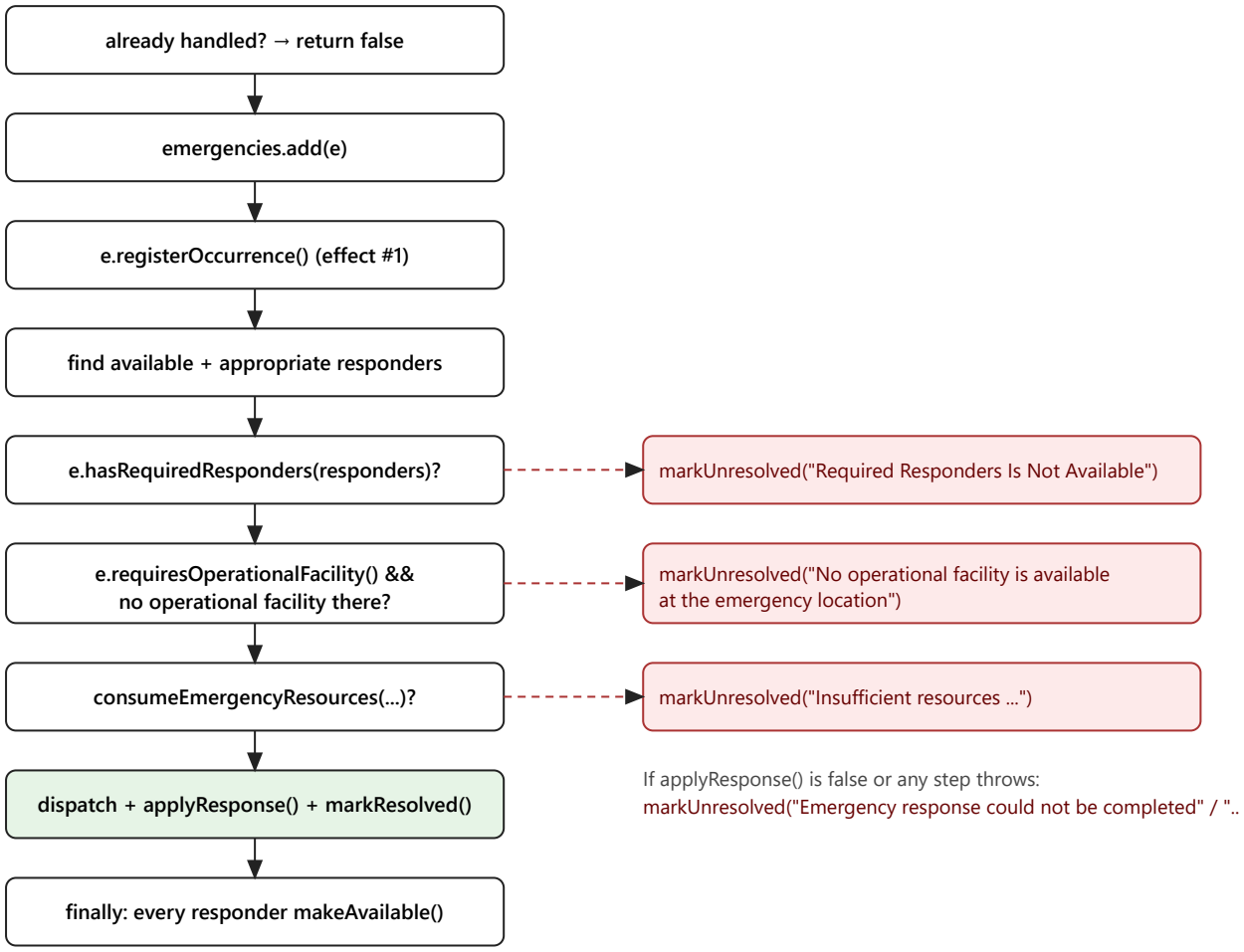


Figure 5 - emergency handling. Failure branches never consume response resources; responders are always released at the end.

- **Responders:** `getAvailableResponders` loops over all personnel and robots and keeps those with `isAvailable()` **and** `canHandleEmergency(type)`. It never checks concrete types. Anyone on the active mission is unavailable, so emergencies never steal agents from the mission, and emergencies do not interrupt it (spec 9.1).
- **Dispatch:** every selected responder is set unavailable and moved to the emergency location (`setCurrentLocation`) - location never decides *who* can respond (spec 9.1.4), it only changes where they end up.
- **Location type:** a `Facilities` object, or the string "Martian Surface" (rover breakdown) or the patient's location string (medical).
- **Release:** a `finally` block calls `makeAvailable()` on all dispatched responders, even on failure.
- **Not retried:** a second call with the same emergency returns `false` immediately.

> **Weak spot to own up to:** the colony dispatches *all* appropriate responders, not just the minimum needed. It is simple, but a real system would pick the smallest valid team. Also, `consumeEmergencyResources` repeats most of the logic of `prepareMissionStart` - a refactor into one shared helper is an obvious improvement.

8.2 Oxygen leak (the one the spec describes in detail)

Spec step	Code
1. Handled when manually triggered	`colony.handleEmergency(new OxygenLeak(habitat, amount))`
2. Identify the affected habitat	`OxygenLeak.affected_facility` (must be a `Habitat`, enforced in the setter)
3. Reduce oxygen in that habitat	`onOccurrence()` → `habitat.applyOxygenLoss(oxygenLossAmount)` - this is the **first** loss
4. Determine responders	`OxygenLeak.hasRequiredResponders`: at least one `Personnel` **and** one `RoboticUnit` that can handle OXYGEN_LEAK (MECHANICAL person, EQUIPMENT_REPAIR robot)
5. Dispatch	Colony sets them unavailable and moves them to the habitat's location
6. Consume resources	1.0 SPARE_PARTS via `consumeEmergencyResources` (all-or-nothing)
7. Repair if possible	`applyResponse()` sets `repaired = true`; `markResolved()`
8. Update habitat state	Oxygen already reduced; responders released

> **The "double cost" rule - say it clearly:** "The oxygen is deducted once when the leak is detected. If the leak is repaired, that is the total loss. If the emergency ends unresolved, `markUnresolved` triggers the `onUnresolved()` hook, which deducts the **same amount again** - so an unresolved leak costs exactly twice. It is a one-time deduction, not a continuous drain, and `deductResource` clamps at zero so a habitat can never go negative."

Numbers (Scenario 4): oxygen 10.0, loss 2.0, spare parts 3.0. Resolved: oxygen 8.0, spare parts 2.0, engineer and robot available again. If the same leak had been unresolved: oxygen 6.0 and spare parts untouched (3.0).

8.3 The other three emergencies - and the requirements we chose

The spec says "you must decide the requirements for the other emergencies and document them". Here they are:

Emergency	Responders required	Other conditions	What happens
Fire in a `Laboratory`	At least one responder who can handle FIRE - a person with skill FIRE_RESPONSE (robots cannot)	none	On occurrence the lab becomes UNAVAILABLE (`onFire`). The response extinguishes it and leaves it DAMAGED. A `Repair` mission or `Laboratory.repair()` restores OPERATIONAL
Rover breakdown on the surface	At least one responder able to repair: a person with MECHANICAL or a robot with EQUIPMENT_REPAIR	none	On occurrence the rover's active mission fails ("Rover breakdown"), the rover is set DAMAGED and unavailable, and stays where it broke down. The response itself does not repair the rover
Medical (patient)	At least one responder with skill MEDICAL	An OPERATIONAL facility at the patient's location; 1.0 MEDICAL_SUPPLIES	After treatment the patient's health status becomes "Stable"

> **Be honest about what is simplified:** the specification lists more consequences for fire (equipment damaged, extra oxygen use, medical help for people in the lab) and says a repair operation "may then be created" after a rover breakdown. The code implements the laboratory status chain and the mission failure, but not those extras. Health is a text status ("Stable"), not a number, so "health increases" is represented by the status change.

8.4 Emergency status machine

`PENDING` (at creation) → `RESOLVED` (everything worked) or `UNRESOLVED` (responders missing, facility missing, resources insufficient, or the response failed or threw). Both are final; unresolved ones are not retried. `markResolved` clears the reason; `markUnresolved` requires a non-empty reason.

9. Simulation and time

```
public static void advanceTime(Colony colony, int timeUnits) {
    for (int i = 0; i < timeUnits; i++) {
        time++;                      // global clock (static)
        colony.advanceTimeUnit();    // let the colony react to one unit
    }
}
```

`Colony.advanceTimeUnit()`:

1. If there is no active mission, return.
2. Ask the environment for a failure reason for the active mission; if one comes back, fail the mission now and stop.
3. Otherwise call `activeMission.advanceTimeUnit()`: if INPROGRESS, each agent spends 1 battery unit (a robot with less than 1 unit fails the mission with "An assigned agent ran out of battery"), `elapsed++`, and the mission completes when `elapsed ≥ needed`.

Spec says time may cause ...	Implemented?
Missions to progress	Yes
Batteries to decrease	Yes (robots, 1 per unit while the mission is in progress)
Environmental conditions to be applied / to change	Applied every unit; the condition itself is changed manually (code or GUI drop-down)
Fatigue to increase	**No** - fatigue is a stored value only
Resources to change	**No** - `PowerStation.generatePower()` exists but the simulation does not call it
Emergencies to occur	**No** - they are triggered manually (the spec says "when manually triggered" for the oxygen leak)

> **Design weakness:** `Simulation.time` is a `public static int` - global mutable state. It is simple, but it means two colonies share one clock and tests must call `resettime()`. A better design is a `Simulation` instance that owns the clock and the colony.

10. The five UML diagrams - how to explain each one

All five are in `mars colony (4).drawio` (open at diagrams.net). The specification says you must "be prepared to explain the purpose of each diagram, justify your design decisions, and demonstrate how the UML design corresponds to the Java implementation".

Draw.io page	Diagram type	One-sentence purpose
`Page-1`	Class diagram	The static structure: classes, interfaces, enums, attributes, methods, relationships
`Object Diagram`	Object diagram	A snapshot of real objects with real values at one moment
`State Machine Diagram`	State machine	The lifecycle of a mission (plus emergency and facility status)
`Sequence A / B / C`	Sequence diagrams	How objects collaborate over time for three operations
`Use Case Diagram`	Use case diagram	What the system offers to external actors

10.1 Class diagram (Page-1)

Say: "It shows the five hierarchies (Personnel, RoboticUnit, Facilities, Mission, Emergency), the `Agent` interface realised by Personnel and RoboticUnit, the `Colony` that aggregates the parts, the enums, and the composition between Habitat and LifeSupportSystem."

Point at: hollow triangle (inheritance), dashed hollow triangle (realization of `Agent`), hollow diamond at `Colony` (aggregation), filled diamond at `Habitat` (composition), dashed diamond from `Facilities` to `Equipment` (aggregation).

> **Tidy-up before the discussion.** Some labels in the class diagram contain typos: *habitate*, *facilties*, *Mars enviroment*, *Roverbreakdown*, *medicalemergency*. Class names in the code are `Habitat`, `Facilities`, `MarsEnvironment`, `RoverBreakdown`, `medicalEmergency`. An examiner who is checking "does the diagram match the code" may notice; either fix them or be ready to say they are label typos.

10.2 Object diagram

It contains **two snapshots**, both taken from the demo scenarios, with values the code really produces:

- **Snapshot A (Scenario 1, just after the mission is accepted):** `colonyAlpha : Colony` links to `scientist101 : Scientist`, `rover1 : ExplorationRover` (with `surveyKit : Equipment`), `habitatA : Habitat` (owning `lifeSupportA : LifeSupportSystem`), `exploration1 : Exploration` and `marsEnv : MarsEnvironment`.
- **Snapshot B (Scenario 4, after the leak):** `leak1 : OxygenLeak` with status RESOLVED, `habitatD` with oxygen 8.0 and spare parts 2.0, engineer and robot available again.

Explain the consistency: In snapshot A the mission state is ASSIGNED, so both agents are already `availability = false` and point to the mission, but oxygen is still 10.0 because resources are only deducted at *start*. In snapshot B oxygen dropped by exactly the leak amount once (10 → 8) and one spare part was consumed (3 → 2).

Arrow notation used: hollow diamond at the whole = aggregation; filled diamond = composition; open arrow = association (mission → agents, mission → facility, colony → environment).

> **If asked "why is an object diagram useful?":** "The class diagram says a Mission has a list of Agents; the object diagram shows *which* agents, in what state, and proves the model can represent a legal real situation - for example that an ASSIGNED mission has unavailable agents."

10.3 State machine diagram

Three machines are drawn; the first is the required one.

1. **Mission lifecycle:** initial dot → NEW → ASSIGNED → IN_PROGRESS → COMPLETED, with FAILED reachable from NEW (rejection), ASSIGNED and IN_PROGRESS; self-loop on IN_PROGRESS for each time unit; a back-edge ASSIGNED → NEW for `unassign`. Two final states.
2. **Emergency status:** PENDING → RESOLVED or UNRESOLVED (both final).
3. **Facility operational status** (laboratory fire cycle): OPERATIONAL → UNAVAILABLE → DAMAGED → OPERATIONAL, plus an admin transition to DESTROYED.

Points the spec requires you to show (and where): rejected mission goes straight NEW → FAILED and never active (dashed edge from NEW); in-progress failure causes (environment, battery, breakdown) on the IN_PROGRESS → FAILED edge; failed missions are not retried (FAILED is final; note box).

10.4 Sequence diagrams

Page	Participants	Story to tell
A - Mission	Operator, Simulation, Colony, Mission, Agents, MarsEnvironment, Facilities	`submitMission` → validation fragment (availability, capability, team, environment, resources) → **alt** accepted (setColony, activeMission, assign agents, ASSIGNED) / rejected (rejectMission, FAILED, not active) → `startActiveMission` (battery, reserve resources, INPROGRESS or FAILED) → **loop** over time units (environment check, battery, elapsed, COMPLETED, release agents)
B - Oxygen leak	Emergency Operator, Colony, OxygenLeak, Habitat, Engineer, RepairRobot, Facilities	`handleEmergency` → first oxygen loss → responders asked → **alt** responders missing (unresolved + second loss) / **alt** resources missing (unresolved + second loss) / success (dispatch, repair, RESOLVED, release)
C - Environment	Colony Operator, Simulation, Colony, MarsEnvironment, Exploration, Agents	Dust storm: the environment is asked, returns the reason, the mission is rejected and never active; second fragment: a storm arriving mid-mission fails it at the next time unit

Reading tip: a solid arrow is a call; a dashed arrow is a return; a small loop on one lifeline is the object doing something to itself (e.g. `state = ASSIGNED`). Frames: **alt** = alternatives separated by a dashed line, **loop** = repeated, **opt** = optional, **ref** = a grouped block.

10.5 Use case diagram

- **Actors:** Colony Operator, Emergency Operator (the two the spec suggests) and Simulation Clock (secondary actor that advances time automatically).
- **Use cases:** all those the spec lists - manage crew and robots, manage facilities and equipment, monitor resources, generate colony status report, submit/start/monitor missions, advance simulation time, monitor environment, trigger and handle emergencies, inspect outcomes.

- **<<include>> (always):** Submit mission includes Validate agents, Check colony-wide resources, Check environment restrictions. Start mission includes Reserve resources. Advance time includes Progress the active mission. Trigger emergency includes Identify and dispatch responders and Consume response resources.
- **<<extend>> (conditional):** Reject mission (when any check fails) extends Submit mission; Fail active mission (environment, battery, breakdown) extends Advance time; Mark emergency UNRESOLVED (no responders or resources) extends Trigger emergency.

> **Why include vs extend?** "Include = the base use case *always* needs it - submitting a mission always validates agents. Extend = behaviour that happens *only under a condition* - rejecting only happens if a check fails."

10.6 Consistency table (spec section 11.7)

Diagram claim	Code that proves it
Mission has states ASSIGNED, IN_PROGRESS, COMPLETED, FAILED	`Mission_Emergency_States` ; transitions in `Mission.assign / startMission / advanceTimeUnit / finishMission / failMission / rejectMission`
Rejected mission becomes FAILED and never active	`Colony.submitMission` calls `rejectMission` before `activeMission = mission` is ever reached
Only one mission active	Check 2 in `submitMission` (`activeMission != null`)
Habitat composes LifeSupportSystem	`private final LifeSupportSystem lifeSupport` created in `Habitat` constructor
Unresolved leak costs double	`OxygenLeak.onOccurrence()` and `onUnresolved()` both call `applyOxygenLoss`
Dust storm rejects exploration	`MarsEnvironment.getMissionRestrictionReason` + check 7 in `submitMission`
Agents released at the end	`makeAvailable()` loops in `failMission` , `rejectMission` , `finishMission`
Resources all-or-nothing	Deduction plan + `restoreResources` in `Colony`

11. The five demonstration scenarios and a live-demo script

11.1 Run it

```
javac -d out/manual -sourcepath src src/Main.java
java  -cp out/manual Main
```

Expected output (verified) - know this by heart:

#	Setup	What the output proves
1	Scientist + rover (survey kit) + exploration mission (2 units, 2.0 oxygen); habitat has 10 oxygen	Before: state `null`, not active, oxygen 10.0. After: **COMPLETED**, no active mission, oxygen **8.0**, agents available, time = 2
2	An **Engineer** (no SCIENTIFIC skill) submitted for exploration	Accepted **false**; state FAILED; reason "*Agent lacks required capabilities*"; not active; agent still available
3	Habitat holds 1.0 oxygen; mission needs 5.0	Accepted false; FAILED; "*Insufficient resources for mission*"; oxygen 1.0 before and after (never negative)
4	Habitat-D (10 oxygen, 3 spare parts), engineer + repair robot, leak of 2.0	Handled true; status **RESOLVED**; repaired true; oxygen 10 → **8**; spare parts 3 → **2**; both responders available again
5	Environment set to DUST_STORM, exploration mission with a rover	Accepted false; FAILED; "*Dust storm prevents exploration missions from starting*"; not active

11.2 A 5-minute live-demo script

1. **(30 s)** Show the package structure: `colony`, `agents`, `personnel`, `roboticunit`, `facilities`, `missions`, `emergencies`, `marsenvironment`, `simulation`.
2. **(60 s)** Open `Colony.submitMission`. Walk the checks top to bottom. Stress: "no instanceof - the agent, mission and environment each answer."
3. **(60 s)** Run `Main`. Narrate scenarios 1-3: success, wrong agent, not enough oxygen.
4. **(60 s)** Open `OxygenLeak`. Show `onOccurrence` and `onUnresolved`; narrate scenario 4 and the double-cost rule.
5. **(30 s)** Scenario 5; open `MarsEnvironment.getMissionRestrictionReason`.
6. **(60 s)** Switch to the draw.io file: object diagram → state machine → one sequence diagram, matching each to the code you just showed.

11.3 GUI demo path (optional, if they ask to see it)

Run `admingui.AdminGuiMain` (the sample colony loads). **Missions** tab: *New Mission...* → create a draft → *Submit Selected Draft...* → *Start Active Mission*; use the dashboard's *Advance +1 / +5* (or the auto-advance tick box). Change the environment drop-down to DUST_STORM and submit an exploration mission to see the rejection. **Emergencies** tab: *New Emergency...* to trigger an oxygen leak. Watch the *Live Activity Feed* for the log lines. The GUI contains no simulation rules; it calls the same methods as `Main`.

12. Testing

- **Main class (`src/Main.java`)** - the required "testing main class": prints before/after state for the five scenarios.
- **`tests/MissionAssignmentTest`** - four self-checks using a small `TestAgent` (an `Agent` test double, which shows the interface makes the colony testable without real people/robots). All four print **PASSED**:

Test	What it proves
Rollback	If the second agent's assignment fails, the first is unassigned, `activeMission` is null, mission is FAILED
Successful lifecycle	submit → start (INPROGRESS, agent unavailable) → finish(true) → COMPLETED, agent available, no active mission
Failed lifecycle	finish(false) → FAILED, agent released
Fail active mission	`failActiveMission("Test failure")` stores the reason, releases the agent

- **What is not covered:** no JUnit suite; no automated tests for emergencies, environment rules or resource totals beyond the scenarios. Say so honestly and point to the scenarios as the evidence.

Run them: `javac -d out/manual -sourcepath src src/tests/MissionAssignmentTest.java` then `java -cp out/manual tests.MissionAssignmentTest`.

13. Honest limitations and what you would improve

Examiners respect a team that knows its own weak spots. Lead with these before you are asked.

Limitation	Why / how to talk about it	Improvement
Fatigue never increases; resources do not change over time; no automatic emergencies	The spec says time "may cause" these; we implemented mission progress, batteries and the environment, and kept emergencies manual as the leak rule says "when manually triggered"	Fatigue per mission unit; call `PowerStation.generatePower()` each unit; a random emergency generator
Fire: only the laboratory's status changes	Equipment damage, extra oxygen use, medical help for occupants are not applied	Damage the lab's equipment in `onOccurrence`; raise oxygen use; create `medicalEmergency` objects for occupants
Rover breakdown does not auto-create a repair mission	The rover is not `Equipment` or `Facilities`, which is what `Repair` accepts	Let `Repair` accept a `RoboticUnit`, or add a rover-repair mission type
Health is a text string	Spec asks for health "to increase"; we model it as status `"Stable"`	Numeric health 0-100 with an increase on treatment
All appropriate responders are dispatched	Simple; minimal team selection is a refinement	Choose the smallest valid set
Duplicated allocation code (`prepareMissionStart` and `consumeEmergencyResources`)	Both implement first-fit all-or-nothing	Extract a single `ResourceAllocator` / helper
`Simulation.time` is static global state	Simple, but ties all colonies to one clock	Instance-based simulation
Mission is accepted but not started automatically	Deliberate: ASSIGNED vs IN_PROGRESS are separate states in the spec	Optional auto-start flag
No printable status report method	Dashboard and console snapshots serve as the report	Add `Colony.generateStatusReport()` returning a formatted string
Naming quirks: `EnviromentStates`, `COMMUNICATION_DISTRIBUTION`, `Mission_Emergency_States`, lower-case `medicalEmergency`, `checkresources`	Historical; renaming touches many files	IDE refactor-rename to Java conventions
`CommunicationStation` and `LifeSupportSystem` exist but are not used by any rule	They model the structure and relationships the spec asks for	Make comms loss depend on the station; make leaks affect `functional`

14. Expected questions and model answers

Answers are written to be said in your own words. Where a question has a "trap", it is flagged.

A. Big picture and design

Q1. Explain your project in one minute.

MARS-OPS simulates a Mars colony. A `Colony` object coordinates personnel, robots, facilities, missions and emergencies. Missions and emergencies are abstract classes with subclasses that define their own rules and state machines. The colony never checks concrete types; it asks `agent.canPerform(mission)`, `emergency.hasRequiredResponders()` and `environment.getMissionRestrictionReason()`. Time advances through `Simulation.advanceTime`. `Main` demonstrates five scenarios.

Q2. Why is there a Colony class? What does it do that others cannot?

The rules involve several objects at once: one active mission across the whole colony, agent availability colony-wide, resources summed across facilities, and dispatching responders from all personnel and robots. No single mission, facility or person has that overview, so the colony owns the lists and enforces cross-object rules. It does *not* own domain knowledge (what a mission needs, what an emergency does) - that stays in the subclasses.

Q3. What is the Agent interface for? Why not make Personnel and RoboticUnit share a parent class?

Humans and robots have different data (skills and fatigue versus battery and capabilities) so a common parent class would be unnatural. They do share a *behavioural contract* - can you perform this mission, can you handle this emergency, are you available. An interface captures exactly that, so `Colony` and `Mission` can treat both as `Agent`. This is the "capability" design the spec's section 6.1 asks for.

Q4. Why don't you just use instanceof to check whether an Engineer can repair?

It would spread type checks across the colony and break whenever a new type is added (open/closed principle). Instead each agent answers for itself. For example, `Engineer.canPerform` returns true for a `Repair` mission if the engineer has the MECHANICAL skill. Note that the colony may still need team-level knowledge - that is asked of the mission (`hasValidTeamComposition`), not hard-coded in the colony.

Q5. How would you add a new mission type, for example Mining?

Create `Mining extends Mission`; pass duration, resources and required capabilities to the super constructor; override `hasValidTeamComposition` (and `onSuccessfulCompletion` if there is an outcome). Update the relevant agents' `canPerform` to accept `Mining`. `Colony`, `Simulation` and the state machine do not change. For a new emergency, extend `Emergency` and implement `applyResponse` and any hooks; add a value to `EmergencyType` and teach agents in `canHandleEmergency`.

Q6. Where did you use abstract classes and where interfaces, and why?

Abstract classes where subclasses share state and code: `Personnel`, `RoboticUnit`, `Facilities`, `Mission`, `Emergency`. **Interface** where unrelated classes share only a behaviour contract: `Agent`. Enums are used for every closed set of values (states, skills, capabilities, resources) instead of strings.

Q7. Show me the four OOP principles in your code.

Encapsulation: private fields, validation in constructors/setters, getters returning copies, guarded state transitions. **Abstraction:** `Agent` and the abstract classes. **Inheritance:** five hierarchies (e.g. `Engineer extends Personnel`). **Polymorphism:** `agent.canPerform()`, `mission.hasValidTeamComposition()`, `emergency.applyResponse()` called on parent references run subclass code.

Q8. What design patterns did you use, even if you didn't name them?

Template method: `Emergency.registerOccurrence()` is final and calls the hooks `onOccurrence()`; `Mission.finishMission` calls `onSuccessfulCompletion()`. **State** (lifecycle held as an enum with guarded transitions). **Strategy-like polymorphism** for team rules and responder rules. **Dependency on abstraction** via `Agent`. I would not claim more than that.

B. UML relationships

Q9. What is the difference between aggregation and composition? Give an example of each from your project.

Composition = strong ownership; the part's lifetime is tied to the whole. `Habitat` creates its own `LifeSupportSystem` in its constructor as a `final` field; it is never passed in. **Aggregation** = a whole made of parts that exist independently. `Colony` holds personnel, robots, facilities, missions; they are created outside and added with `addPersonnel` etc., and could be reused elsewhere.

Q10. Why is the association between Mission and Agent not aggregation?

A mission does not "consist of" agents in a whole-part sense; it temporarily *works with* them, and the agents are released when the mission ends. Both sides exist independently, which is the definition of association. The link is also two-way in code: `Mission.agents` and `Personnel.mission` / `RoboticUnit.mission`.

Q11. Why is the Facilities-to-Equipment relationship aggregation?

Equipment can be created separately, moved, or used by a robot (rovers also carry equipment). A facility is a collection of equipment but does not own its lifecycle. So hollow diamond, dashed in my class diagram.

Q12. What does the arrow into the Agent interface mean?

A dashed line with a hollow triangle is *realization*: `Personnel` and `RoboticUnit` implement the interface. Solid line with hollow triangle is inheritance (`extends`).

Q13. Why is the oxygen-leak emergency associated with Habitat but Fire with Laboratory?

The setters enforce it: `OxygenLeak.setAffected_facility` throws unless the facility is a `Habitat`; `Fire.setAffected_facility` throws unless it is a `Laboratory`. This matches the spec (an oxygen leak occurs inside a habitat; a laboratory catches fire).

C. Missions

Q14. Walk me through what happens when I call submitMission.

Eight ordered checks: agents provided, no other mission active, each agent non-null/unique, available, can perform; the mission's team rule; environment restriction; colony-wide resources. The first failure calls `rejectMission(reason)` - the mission becomes FAILED with a reason, is recorded, and is never active. If all pass: `setColony`, `activeMission = mission`, `assign` each agent (agents become unavailable), state ASSIGNED, return true. If an assignment throws, the colony rolls back.

Q15. What are the mission states and what moves a mission between them?

(null/NEW) → ASSIGNED on acceptance; ASSIGNED → INPROGRESS on `startActiveMission()`; INPROGRESS → COMPLETED when elapsed time reaches the duration; FAILED from rejection, a failed start, environment, battery exhaustion, rover breakdown or a forced failure. COMPLETED and FAILED are final.

Q16. Why does ASSIGNED not start automatically?

The spec defines ASSIGNED and IN_PROGRESS as separate states and says time counts "when it becomes IN_PROGRESS". We made the start explicit so the operator can see the assigned state, and resources/battery are checked at that moment. It is a one-line change to auto-start if preferred.

Q17. How do you guarantee only one mission is active?

The colony has a single field `activeMission`. `submitMission` rejects any new mission while it is non-null. It is cleared by every terminal path through `Colony.clearActiveMissionIfTerminal`, which missions call themselves when they complete or fail.

Q18. What happens to the agents when a mission ends or fails?

Every terminal method (`finishMission`, `failMission`, `rejectMission`) loops over the assigned agents calling `makeAvailable()`, clears the list, and notifies the colony. This satisfies "all assigned agents become available again when the mission ends".

Q19. Can a failed mission be retried?

No. `submitMission` returns false immediately for a mission whose state is not null, and `failMission` only works from ASSIGNED or INPROGRESS. To try again you create a new mission object (spec 6.2 rule 6).

Q20. What are the requirements for each mission type?

Exploration: a SCIENTIFIC agent plus a rover with SURFACE_TRAVEL + EQUIPMENT_CARRYING and operational equipment. **Repair:** a MECHANICAL agent plus an agent with EQUIPMENT_REPAIR; the target must be DAMAGED. **Research:** a SCIENTIFIC agent and an operational laboratory with operational scientific equipment. **Rescue:** a COMMUNICATION-skilled person plus a vehicle (SURFACE_TRAVEL or ACCESS_INACCESSIBLE_AREAS). Robots also need battery ≥ duration and the mission's required capabilities.

Q21. What is assigningAgent in Mission? It looks odd.

It is a re-entrancy guard. `Mission.assign(agent)` calls `agent.assignMission(this)`; if someone calls `agent.assignMission(mission)` directly, the agent calls `mission.assign(this)` so the mission always stays the single authority. Without a guard those two calls would call each other forever. `assigningAgent` tells the agent "the mission is already handling this assignment, just record it".

Q22. What does the rollback in submitMission do and why is it needed?

A mission with several agents is assigned one by one. If the third assignment throws, the first two agents would be left unavailable and linked to a half-assigned mission. The colony catches the exception, unassigns those already assigned, clears `activeMission`, and rejects the mission with the exception message. The `testRollback` check covers this.

Q23. Why is state null before acceptance rather than an enum value?

It reflects "not yet part of the colony's workflow" and lets guards like `rejectMission` require `state == null`. It is a design compromise; a `NEW` value would be cleaner and is a good improvement. The state machine diagram draws it as NEW.

D. Resources

Q24. How is the colony-wide total computed?

`Colony.getColonyResourceTotal()` loops over facilities, skips any that are not OPERATIONAL, and merges each facility's resource map with `Double::sum`. Resources in damaged, unavailable or destroyed facilities do not count.

Q25. Explain your resource allocation policy.

First-fit in the order facilities were added. For each required resource the colony takes as much as possible from the first operational facility, then the next, and so on. It first builds a plan without changing anything; only if the whole request is covered does it deduct.

Q26. What does "all-or-nothing" mean, and how did you implement it?

Either the full required quantity is supplied or nothing is consumed. We create a `deductionPlan` and a `remaining` map; if any entry in `remaining` is still positive we return false before touching any facility. During the real deduction, if a deduction returns less than planned or throws, `restoreResources` puts back the saved original maps.

Q27. Example: Habitat A has 300 oxygen, Storage has 200. A mission needs 450. And 600?

450: plan takes 300 from Habitat A and 150 from Storage → valid; Storage keeps 50. 600: after taking all 500, 100 remains uncovered → nothing is deducted and the operation fails with a clear reason. (The submission check uses the colony total 500 < 600, so it is rejected earlier.)

Q28. Can resources become negative anywhere?

No. `Facilities.deductResource` removes `Math.min(available, requested)`, so a leak larger than the stock leaves exactly zero. Mission and emergency resource use go through the all-or-nothing plan, and constructors reject negative or non-finite amounts.

Q29. Why are mission resources deducted at start, not at submission?

Submission only decides whether the mission is acceptable (resources are checked). An assigned mission that never starts should not burn resources; consumption belongs to execution. `resourcesPreparedMissions` ensures a mission is charged only once. If stock disappeared before start, the mission fails with "Insufficient resources to start mission".

E. Emergencies

Q30. Walk me through handling an oxygen leak.

`handleEmergency(leak)`: record it; `registerOccurrence()` deducts the leak amount from that habitat (first loss); find available responders who say they can handle OXYGEN_LEAK; `OxygenLeak.hasRequiredResponders` needs at least one human and one robot; consume 1.0 spare parts all-or-nothing; dispatch (set unavailable, move to the habitat); `applyResponse()` repairs; `markResolved()`; release responders. If responders or resources are missing it is marked UNRESOLVED with a reason and oxygen is deducted a second time.

Q31. Why does an unresolved leak cost twice as much oxygen?

The specification says so: the loss is deducted when the leak is detected; if repaired that is the total; if unresolved the same amount is deducted again. We implement it with the template hooks: `onOccurrence()` deducts once, `onUnresolved()` deducts again and runs only the first time the emergency becomes unresolved. It is one-time, not a continuous drain.

Q32. What if there are no responders available?

The emergency is marked UNRESOLVED with the reason "Required Responders Is Not Available"; no response resources are consumed (the resource step is never reached); it is not retried; for a leak the second oxygen deduction happens.

Q33. What requirements did you choose for the other three emergencies?

Fire: a responder with FIRE_RESPONSE. **Rover breakdown:** a responder able to repair (MECHANICAL person or EQUIPMENT_REPAIR robot). **Medical:** a responder with MEDICAL, an operational facility at the patient's location, and 1.0 medical supplies. They are documented in the README.

Q34. Do emergencies interrupt the active mission? Can they use the mission's agents?

No and no. Emergencies are not limited by the one-mission rule, but responders must be available, and agents on the active mission are unavailable. The only emergency that touches a mission is a rover breakdown, which fails the mission that rover is on, by design (it is the cause of the failure).

Q35. Does location restrict who can respond?

No (spec 9.1.4). Any available agent that reports it can handle the emergency type is dispatched; dispatching moves it to the affected location with `setCurrentLocation`.

Q36. A rover breaks down. What changes?

`RoverBreakdown.onOccurrence()`: if the rover has a mission, `failMission("Rover breakdown")`; the rover's health becomes DAMAGED and it becomes unavailable. It stays where it was. The colony then looks for repair-capable responders. Limitation: we do not automatically create a repair mission.

Q37. How do fire consequences appear in your model?

The laboratory goes UNAVAILABLE with `onFire = true` (so it no longer counts for colony totals or Research missions); after the response it is DAMAGED, and a Repair mission or `repair()` returns it to OPERATIONAL. Equipment damage, oxygen increase and treating occupants are not implemented - say that directly.

Q38. Why is the emergency's applyResponse separate from resource consumption and dispatch?

Colony-level concerns (who is available, resource totals, moving agents) are the same for every emergency, so the colony does them once. What the response *does* differs per emergency, so each subclass implements `applyResponse`. That is the single-responsibility split.

F. Environment and time

Q39. How does a dust storm affect the colony?

At submission, an exploration mission that uses a rover is rejected with "Dust storm prevents exploration missions from starting". On every time unit the colony asks the environment whether the active mission must fail; a mission with a SURFACE_TRAVEL agent fails with "Dust storm caused the active rover mission to fail".

Q40. What happens to missions already in progress when the environment changes?

Nothing immediately. On the next call to `advanceTimeUnit` the colony asks the environment about the active mission before progressing it. If a reason is returned the mission is failed at once and its agents released; if not, it continues. This is documented in the README as the spec requires.

Q41. How does communication disruption work?

Missions that use a drone (AERIAL_RECONNAISSANCE) are rejected - "remote drones cannot be controlled" - as are Rescue missions that depend on a communication-skilled person. An active mission with a drone, or a Rescue with a communication agent, fails at the next time unit.

Q42. Why does the environment return a reason string instead of failing the mission itself?

It keeps responsibilities separate: the environment knows *what* is forbidden; the mission and colony own *state changes*. Returning a reason lets the colony reuse it as the stored failure reason and keeps `MarsEnvironment` free of mission-lifecycle code.

Q43. What does one time unit do?

`Simulation.advanceTime` increments the clock and calls `colony.advanceTimeUnit()`: environment check; if the active mission is IN_PROGRESS every robot loses 1 battery (failing the mission if it cannot), `elapsed` increases, and the mission completes at the duration. Fatigue and resource changes are not simulated.

G. Code quality and tricky points

Q44. Why is Simulation static? Is that a good design?

It was the simplest way to expose a shared clock (`Simulation.advanceTime(colony, n)`), and the spec accepts "an equivalent design". The downside is global state: all colonies share one counter and tests must call `resettime()`. A better design is an instance that owns the clock and the colony.

Q45. Some names have spelling mistakes (EnviromentStates, COMMUNICATION_DISTRIBUTION, medicalEmergency). Why?

They were chosen early and used throughout the code, the GUI and the diagrams; renaming them is a mechanical IDE refactor but would touch many files late in the project. We kept names consistent between code and UML rather than risk breaking either. `COMMUNICATION_DISTRIBUTION` means communication *disruption*.

Q46. You dispatch all available responders. Is that right?

It is a simplification. It always satisfies the requirement and keeps the colony code short, but a better version would select the smallest valid team and leave other agents free. We would implement that as a method on `Emergency` that picks responders from the candidate list.

Q47. Is there duplicated code?

Yes: `prepareMissionStart` and `consumeEmergencyResources` both implement the same first-fit all-or-nothing plan. They should share one helper method. It is a refactoring we would do first.

Q48. What happens if I give a null or invalid value, for example a negative fatigue?

Constructors and setters throw `IllegalArgumentException` (for example fatigue outside 0-100, blank names, null skills, non-finite resource amounts). State-machine violations throw `IllegalStateException` (failing a COMPLETED mission). The colony converts assignment exceptions into a rejected mission rather than crashing.

Q49. How do you know the diagrams match the code?

The object diagram values come straight from running the demo scenarios (oxygen 10 → 8, spare parts 3 → 2). The state machine is built from `Mission`'s guarded methods; the sequence diagrams list the real method names called in `Colony.submitMission`, `handleEmergency` and `advanceTimeUnit`. The use cases correspond to features available in the console demo and the GUI.

Q50. What is the hardest bug or design problem you solved?

Good candidates, all real: (1) the circular assignment between `Mission.assign` and `Agent.assignMission`, solved with the `assigningAgent` guard; (2) making resource consumption atomic across several facilities with a plan-then-commit approach and a restore path; (3) deciding where the "double oxygen loss" belongs, solved with template-method hooks so only `OxygenLeak` knows about it. Pick the one you personally worked on and describe your part.

Q51. What did each team member do? (they may ask each member to explain a part)

Prepare this in advance: the policy says **every member must understand the whole system**, including parts they did not write. Make sure each person can walk through `submitMission`, the mission state machine, the oxygen leak and one UML diagram. Do not say "I only did the GUI".

Q52. If I change X right now, can you do it? (typical live-change requests)

- **Change the oxygen loss to 5:** pass 5.0 to `new OxygenLeak(habitat, 5.0)`.
- **Make unresolved leaks cost three times:** in `OxygenLeak.onUnresolved` call `applyOxygenLoss(oxygenLossAmount * 2)` (so 1 + 2 = 3 times total).
- **Change the allocation policy to "fullest first":** sort the facility list by the amount of the resource before building the plan in `prepareMissionStart` / `consumeEmergencyResources`.
- **Make drones unaffected by comm disruption:** remove the `usesRemoteDrone` branch in `MarsEnvironment`.
- **Auto-start accepted missions:** call `startActiveMission()` at the end of a successful `submitMission`.
- **Add a new state:** add to `Mission_Emergency_States` and update the guards in `Mission`; update the state diagram.

15. Cheat sheet

Class → one-line job

`Colony`	Coordinator: one active mission, totals, allocation, responders
`Agent`	Interface: canPerform, canHandleEmergency
`Mission`	Lifecycle + team rule + duration
`Emergency`	Status + responder rule + effect hooks
`MarsEnvironment`	Answers: restriction / failure reason
`Facilities`	Location, status, resources, equipment
`Simulation`	Clock: advanceTime(colony, n)

Numbers and strings to remember

Battery cost	1.0 per time unit per robot
Leak spare parts	1.0 SPARE_PARTS
Medical supplies	1.0 MEDICAL_SUPPLIES
Fatigue range	0 - 100
Scenario 1 oxygen	10.0 → 8.0, time = 2
Scenario 4 oxygen	10.0 → 8.0; spare parts 3 → 2
Unresolved leak	2 × the leak amount

Rules in one line each

- One active mission; a rejected mission is FAILED with a reason and never active; failed missions are never retried.
- ASSIGNED → (`startActiveMission`) → INPROGRESS → COMPLETED/FAILED; agents released at the end.
- Totals count only OPERATIONAL facilities; allocation is first-fit by facility order; all-or-nothing; never negative.
- Mission resources: checked at submission, deducted at start (once).
- Emergencies: do not interrupt missions; responders must be available; unresolved → reason, no resources consumed, no automatic retry.
- Dust storm blocks rover exploration and fails active rover missions; comm loss blocks drones and communication rescues.
- Environment change affects an in-progress mission at the **next** time unit.

Submission check order

agents present → no active mission → each agent (non-null, unique, available, canPerform) → team composition → environment → resources.

Commands

```
javac -d out/manual -sourcepath src src/Main.java            java -cp out/manual Main
javac -d out/manual -sourcepath src src/admingui/AdminGuiMain.java   java -cp out/manual
admingui.AdminGuiMain
javac -d out/manual -sourcepath src src/tests/MissionAssignmentTest.java   java -cp out/manual
tests.MissionAssignmentTest
```

> **Final advice.** Speak from the code: open `Colony.submitMission`, `Mission`, `OxygenLeak` and `MarsEnvironment` and trace one example end to end. Say what you did *not* implement before they ask. If you do not know an answer, say how you would find it in the code - that is what the examiner is checking.